



**FACULTY OF INFORMATION TECHNOLOGY**

**SEMESTER: 202509**

**SUBJECT CODE: BIT1123**

**SUBJECT NAME: OBJECT ORIENTED PROGRAMMING**

**LEVEL: BACHELOR**

**Course Title : SELF-REFLECTIVE REPORT**

**LECTURER NAME: Dr. NAZMIRUL IZZAD BIN NASSIR**

**ASSIGNMENTS TYPE: ASSIGNMENT 1 (Individual)**

**TOTAL MARKS %: (20%)**

**PREPARED BY:**

| No. | Name                      | ID           | PROGRAMME |
|-----|---------------------------|--------------|-----------|
| 3.  | ABDELRHMAN AHMED IBRAHEEM | 202505010324 | BCSSE     |

# SELF-REFLECTIVE REPORT

## INTRODUCTION

The Object-Oriented Programming (OOP) paradigm is the bedrock of modern software development, allowing developers to create software that is modular, reusable and maintainable. Throughout this semester, I embarked on a learning journey that traversed ten hands-on tutorials, moving beyond Java basics to grasp complex concepts like inheritance, encapsulation, abstraction, file handling and GUI development.

Furthermore, this learning process not only involved writing code but also familiarizing myself with industry-standard practices such as using version control systems like Git, and managing projects with GitHub, which are indispensable in the professional software landscape. This report serves as an account of my learning experience, outlining the knowledge I've gained, the challenges I encountered, the strategies I employed to overcome them and my plans for continued learning.

## KNOWLEDGE GAINED FROM EACH TUTORIAL

Here's a detailed week-by-week recap of the knowledge acquired, connecting code examples to theoretical principles:

### **Week 1 - Fundamentals and Control Flow:**

- Learned the basic structure of a Java program, including the class keyword and the main method.
- Mastered the use of the Scanner class to accept user input from the console.
- Applied decision-making statements (if / else if / else) to calculate student grades based on their scores and to handle invalid inputs gracefully.

### **Week 2 - Classes and Constructors:**

- Understood how to define classes (e.g., Student) that represent real-world entities.
- Learned the importance and function of the constructor in initializing the attributes of objects using the 'this' keyword.
- Implemented methods (e.g., displayInfo(), study()) to encapsulate the behavior of objects.

# SELF-REFLECTIVE REPORT

## **Weeks 3 & 4 - Inheritance and Polymorphism:**

- This was a pivotal point in my OOP learning; I learned to establish class hierarchies using the 'extends' keyword.
- Practiced method overriding using the '@Override' annotation, enabling classes like Student and Lecturer to provide their own implementation of the introduce() method from the parent Person class.
- Gained proficiency in using the 'super()' keyword to call the parent class's constructor and pass shared attributes (name and ID).

## **Week 5 - Encapsulation:**

- Comprehended the significance of data hiding by declaring attributes as 'private'.
- Implemented setter and getter methods (e.g., setCGPA(), getCGPA()) to control access to attributes, ensuring data integrity and allowing for future validation logic.

## **Week 6 - Deepening Inheritance:**

- Furthered my understanding of inheritance with the introduction of Employee and Lecturer classes.
- Learned how a derived class can leverage parent class attributes while adding new ones (subject, department), demonstrating the power of code reuse.

## **Week 7 - Abstraction using Abstract Classes:**

- This was another mind-opening session. I learned that abstract classes cannot be instantiated but serve as blueprints for other classes.
- Implemented the abstract method operate() in the Appliance class, compelling subclasses (Refrigerator, WashingMachine) to provide their specific implementations, thus enforcing a common contract.

## **Weeks 8 & 9 - File Handling, Data Structures, and Exceptions:**

- Acquired knowledge of using ArrayList for dynamic storage of data.
- Practiced file input/output operations using FileWriter and Scanner to read from and write to task.txt.
- A major learning was exception handling using try-catch blocks, particularly for FileNotFoundException and IOException, which significantly improved code robustness and prevented crashes.

# SELF-REFLECTIVE REPORT

## Week 10 - GUI and Event-Driven Programming:

- This culminated in a practical application of many concepts, using the Swing library to build an interactive graphical window (JFrame).
- Learned the principles of event-driven programming by implementing the ActionListener interface, enabling buttons to respond to user interactions, validate answers, and provide immediate feedback.

## CHALLENGES ENCOUNTERED

Several technical and conceptual hurdles presented themselves during this learning process, with the most prominent being:

- **Understanding super() in Inheritance (Weeks 3/4):** Initially, I struggled to comprehend why calling the parent class's constructor was a requirement and how to correctly pass arguments to it.
- **Encapsulation vs. Direct Access (Week 5):** For small programs, I questioned the necessity of private variables with getters/setters versus directly accessing public variables.
- **Abstract Classes vs. Interfaces (Week 7):** Visualizing when to use an abstract class versus a regular class proved challenging, and I found myself confused about the distinction between abstract classes and interfaces (which we have not yet covered).
- **File Paths (Weeks 8/9):** I repeatedly encountered the FileNotFoundException because I incorrectly placed the task.txt file in my project structure or had a misunderstanding of relative versus absolute paths.
- **GUI and Event Handling (Week 10):** The most complex aspect was grasping how ActionListener works, identifying which button was clicked, and how information was passed between objects (e.g., how the GUI retrieved the question text from the Questions object).

## HOW THE CHALLENGES WERE OVERCOME

These challenges were not roadblocks but rather stepping stones that propelled deeper learning. Here's

# SELF-REFLECTIVE REPORT

## how I navigated them:

- **Employed Debugging Tools:** I consistently used the breakpoint functionality in my IDE (IntelliJ IDEA/Eclipse) to step through my code line by line, observing the flow of execution and variable values between objects.
- **Revisiting Theory and Visual aids:** To cement my understanding of inheritance and abstraction, I reread my lecture notes multiple times and watched relevant video tutorials. I also sketched out simple UML diagrams to illustrate class relationships.
- **Hands-on Experimentation:** Rather than simply copying solutions, I actively rewrote the exercises, adding new features to reinforce my grasp. For example, I added a new Microwave class to the Appliance example (Week 7) to test my understanding of abstract classes.
- **Mastering File Paths:** I learned to use relative paths tied to the project root and incorporated `System.getProperty("user.dir")` to display the current working directory, which permanently resolved the `FileNotFoundException`.
- **Deconstructing Event-Driven Logic:** I meticulously examined the code for the QuizBattleGUI, realizing that the `getSource()` method of the `ActionEvent` object provided a reference to the button that was clicked, allowing for comparison with the answer.

## IMPROVEMENTS IN JAVA PROGRAMMING SKILLS

### My proficiency in Java programming has evolved significantly in various aspects:

- **Code Structure and Clarity:** I've moved from writing monolithic code in a single main method to distributing responsibilities across multiple classes and methods with meaningful names.
- **Object-Oriented Design:** I now approach problems by identifying and manipulating objects (students, employees, appliances) rather than thinking in terms of sequential procedures.
- **Defensive Programming:** Writing code that anticipates and handles potential errors (like missing files or incorrect input) using try-catch blocks has become second nature.
- **Project Organization:** I've learned to structure my projects logically, separating business logic from user interface code (Separation of Concerns).

# SELF-REFLECTIVE REPORT

## UNDERSTANDING OF OBJECT-ORIENTED PROGRAMMING (OOP) CONCEPTS

The four pillars of OOP are no longer abstract concepts but are deeply integrated into my practical understanding:

- **Encapsulation:** The Student object effectively hides its internal data and provides public methods (getters/setters) to control access to it (Week 5).
- **Inheritance:** The class hierarchies, such as Person-Student/Lecturer, Employee-Lecturer, and Appliance-Refrigerator/WashingMachine, demonstrate the power of reusing existing code (Weeks 3/4, 6).
- **Polymorphism:** Method overriding (introduced with introduce() in Weeks 3/4 and operate() in Week 7) allows objects of different classes to be treated uniformly while exhibiting distinct behaviors.
- **Abstraction:** The abstract class Appliance hides the implementation details of how different devices operate while establishing a clear contract for their functionality.

## FUTURE LEARNING PLANS

These tutorials are merely the first step in my journey toward becoming a proficient Java developer. My future learning trajectory includes:

- **Advanced Data Structures:** Expanding my knowledge beyond ArrayList to include structures like HashMap, TreeSet, and efficient searching and sorting algorithms.
- **Industry Frameworks:** Venturing into frameworks such as Spring Boot and Hibernate to build web applications and microservices.
- **Concurrency:** Exploring threads and multithreading to enhance application performance.
- **Unit Testing:** Learning JUnit for automated testing to ensure code quality and prevent regressions.
- **Professional Repository Management:** Continuing to host my personal projects on GitHub with professional READMEs and utilizing branching strategies like feature/develop branches.

# SELF-REFLECTIVE REPORT

## CONCLUSION

This ten-week long adventure in Java and Object-Oriented Programming provided me with a qualitative leap in my studies and my future career. I progressed from writing basic procedural instructions to developing cohesive applications that can interact with both users and files. This was more than just theoretical lectures; it was a real learning experience that presented many real-world problems which I overcame through determination and a rational method of scientific bug finding.

Collocating these projects in a coherent repository on Github was not just another requirement but a real exercise on how to present myself as a capable and ready programmer for the job market in the future. I am now confident in my understanding of OOP and I'm looking forward to implementing these ideas in more complex programs, which will contribute to well written, scalable and efficient softwares.

## APPENDIX

**GitHub Repository URL:**

[https://github.com/Rahman1997ag/JAVA\\_ASSIGNMENT\\_1.git](https://github.com/Rahman1997ag/JAVA_ASSIGNMENT_1.git)